

Greenplum

Technical Publications

Greenplum Performance Monitor Guide

Last Revised: November 7, 2008 3:19 pm

Contents - Greenplum Performance Monitor Guide

About This Guide 1
Greenplum Performance Monitor Documentation 1
Document Conventions 1
Contact Us 4

Chapter 1: About Greenplum Performance Monitor 5
Greenplum Performance Monitor Architecture 5
Greenplum Performance Monitor Agents 6
Greenplum Performance Monitor Database 6
Current Data Tables 6
Tail Data Tables 7
Historical Data Tables 7
Greenplum Performance Monitor Console 7
Greenplum Performance Monitor Web Service 7

Chapter 2: Installing Greenplum Performance Monitor 9
Preparing for Installation 9
System Requirements 9
Creating the Monitor Superuser 9
Editing postgresql.conf 10
Installing Greenplum Performance Monitor on the Master 10
Installing Greenplum Performance Monitor on the Master 10
Setting up Greenplum Performance Monitor on the Master 11
Installing the Segment Monitor Agents 12
Uninstalling Greenplum Performance Monitor 13

Chapter 3: Administering Greenplum Performance Monitor 15
Starting and Stopping Greenplum Performance Monitor 15
Reconfiguring Greenplum Performance Monitor 15
Editing the Configuration File 15
Reconfiguring a Monitor Instance with gpperfmon 16
Agent Administration 17
Restarting the Master Agent 17
Adding and Removing Monitored Hosts 17
Viewing and Maintaining Master Agent Log Files 17
Greenplum Performance Monitor Database Administration 18
Backing Up and Restoring the Monitor Database 18
Maintaining Historical Data 18
Web Server Administration 18
Starting and Stopping the Web Service 18
Configuring the Web Server 19
Viewing and Maintaining Web Server Log Files 20

Chapter 4: Using Greenplum Performance Monitor 21
About the Greenplum Performance Monitor Console 21
Monitoring the Current System Workload 21
Monitoring Active Queries 22
Setting Thresholds for Notification 22
Viewing Historical System Metrics 22
Viewing Aggregated System Metrics 22

Viewing Historical Query Metrics 24
Viewing Historical Database Metrics 24
Viewing Aggregated Database Statistics 24

Appendix A: Utility Reference: gpperfmon 26

Appendix B: Configuration Reference: gpperfmon.conf 28
Master Agent Options 28
Greenplum Performance Monitor Console Options 29

Appendix C: Greenplum Performance Monitor Catalog Reference 31
system_now 32
system_tail 34
system_history 36
database_now 38
database_tail 39
database_history 40
queries_now 41
queries_tail 42
queries_history 43
iterators_now 44
iterators_tail 49
iterators_history 54

Copyright © 2008 by Greenplum, Inc. All rights reserved.

This publication pertains to Greenplum software and to any subsequent release until otherwise indicated in new editions or technical notes.

Information in this document is subject to change without notice. The software described herein is furnished under a license agreement, and it may be used or copied only in accordance with the terms of that agreement. Upgrades are provided only at regularly scheduled software release dates. No part of this publication may be reproduced, transmitted, or translated in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without the prior written permission of Greenplum, Inc. Greenplum makes no warranty of any kind with respect to the completeness or accuracy of this manual.

Greenplum, the Greenplum logo, and Greenplum Database are trademarks of Greenplum, Inc. PostgreSQL is a trademark of Marc Fournier held in trust for The PostgreSQL Development Group. All other company and product names used herein may be trademarks or registered trademarks of their respective companies.

Some Greenplum Database management tools make use of the Apache Portable Runtime library (http://www.apache.org/licenses) and the Eventlib library Copyright 2000-2004 Niels Provos and are subject to their respective license agreements.

Preface

This guide provides information for system administrators and Greenplum Database super users responsible for administering the Greenplum Performance Monitor.

- About This Guide
- Greenplum Performance Monitor Documentation
- Contact Us

About This Guide

This guide provides information and instructions for installing, configuring and maintaining Greenplum Performance Monitor. This guide is intended for system administrators and database super users responsible for managing the Greenplum system and databases.

This guide assumes knowledge of Linux/Unix system administration, database management systems, database administration, and structured query language (SQL).

Greenplum Performance Monitor Documentation

The Greenplum Performance Monitor documentation is provided in PDF. The documentation can be found in the `docs` directory of your Greenplum Performance Monitor installation, or on the Greenplum Network.

The following documents are provided with this release of Greenplum Performance Monitor:

- **Release Notes** — The release notes (`README`) explain the new features and any known issues associated with this release.
- Greenplum Performance Monitor Guide — This guide contains conceptual, instructional and reference information for database and system administrators responsible for installing, configuring and administering Greenplum Performance Monitor.

Document Conventions

The following conventions are used throughout the Greenplum Database documentation to help you identify certain types of information.

Text Conventions

Table 0.1 Text Conventions

Text Convention	Usage	Examples
bold	Button, menu, tab, page, and field names in GUI applications	Click **Cancel** to exit the page without saving your changes.
italics	New terms where they are defined Database objects, such as schema, table, or columns names	The *master instance* is the postmaster process that accepts client connections. Catalog information for Greenplum Database resides in the *pg_catalog* schema.
`monospace`	File names and path names Programs and executables Command names and syntax Parameter names	Edit the `postgresql.conf` file. Use `gpstart` to start Greenplum Database.
`monospace italics`	Variable information within file paths and file names Variable information within command syntax	`/home/gpadmin/`*`config_file`* `COPY` *`tablename`* `FROM '`*`filename`*`'`
`monospace bold`	Used to call attention to a particular part of a command, parameter, or code snippet.	Change the host name, port, and database name in the JDBC connection URL: `jdbc:postgresql://`**`host:5432/mydb`**
`UPPERCASE`	Environment variables SQL commands Keyboard keys	Make sure that the Java `/bin` directory is in your `$PATH`. `SELECT * FROM my_table;` Press `CTRL+C` to escape.

Command Syntax Conventions

Table 0.2 Command Syntax Conventions

Text Convention	Usage	Examples
`{ }`	Within command syntax, curly braces group related command options. Do not type the curly braces.	`FROM { 'filename' | STDIN }`
`[ ]`	Within command syntax, square brackets denote optional arguments. Do not type the brackets.	`TRUNCATE [ TABLE ] name`
`...`	Within command syntax, an ellipsis denotes repetition of a command, variable, or option. Do not type the ellipsis.	`DROP TABLE name [, ...]`
`|`	Within command syntax, the pipe symbol denotes an "OR" relationship. Do not type the pipe symbol.	`VACUUM [ FULL | FREEZE ]`

`$ system_command`
`# root_system_command`
`=> gpdb_command`
`=# su_gpdb_command`
	Denotes a command prompt - do not type the prompt symbol. `$` and `#` denote terminal command prompts. `=>` and `=#` denote Greenplum Database interactive program command prompts (`psql` or `gpssh`, for example).	`$ createdb mydatabase`
		`# chown gpadmin -R /datadir`
		`=> SELECT * FROM mytable;`
		`=# SELECT * FROM pg_database;`

Naming Conventions and Acronyms

Table 0.3 Naming Conventions and Acronyms

Convention	Meaning
`$MASTER_DATA_DIRECTORY/gpperfmon/`	The base directory for Greenplum Performance Monitor external data tables, configuration, and logs. For example: `/usr/local/greenplum-perfmon-2.0.0.0`
`gpadmin`	The default name for the Greenplum Database super user.
`KB`	Kilobytes (2^{10})
`MB`	Megabytes (2^{20})
`GB`	Gigabytes (2^{30})
`TB`	Terabytes (2^{40})

Contact Us

To contact Greenplum customer support, call 1-866-410-6060 or send email to:

support@greenplum.com

1 About Greenplum Performance Monitor

Greenplum provides an optional performance monitoring feature that administrators can install and enable for Greenplum Database. Greenplum Performance Monitor lets administrators view the status and performance of actively running queries and allows for detailed analysis of historical queries and system metrics. A browser-based graphical interface provides easy access to a variety of configurable views and reports on performance data.

Greenplum Performance Monitor Architecture

Greenplum Performance Monitor is based on an agent architecture. Agent processes are installed on each segment host, and all other monitor components are installed on the master host.

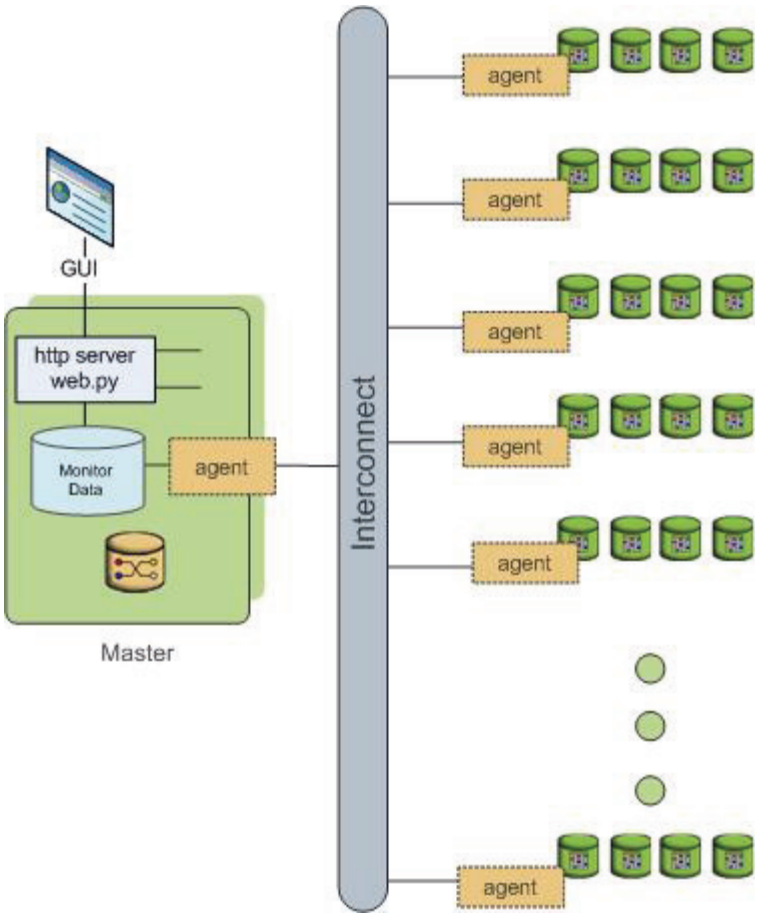


Figure 1.1 Greenplum Performance Monitor Architecture

The agents collect performance data about active queries and system utilization and provide the data to the Greenplum master at regular intervals. The data is stored in a dedicated Greenplum Performance Monitor database on the master, where it can be accessed through the Greenplum Performance Monitor Console or through SQL queries.

Greenplum Performance Monitor Agents

Greenplum Performance Monitor agents run on all hosts in a Greenplum Database array. Each segment contains its own segment agent `gpsmon` process, which collects runtime statistics for system metrics, queries, and iterators. A master agent `gpmmon` process controls and maintains TCP connections to the segment agents.

At runtime, the master agent retrieves performance data from the segment agents at configurable intervals called `quantum`. The master agent pulls together the data from all agents, stores it in flat files, and periodically commits the data in the files to the Greenplum Performance Monitor database.

Greenplum Performance Monitor Database

The Greenplum Performance Monitor database is a distributed Greenplum database dedicated to storing and serving performance data for the system. The monitor installation utility runs SQL cripts to build and initialize this database, named `gpperfmon`, on the master host.

Greenplum administrators can connect to the monitor database to manage and query performance data using client programs such as `psql` or application programming interfaces (APIs) such as JDBC and ODBC. Also, administrators can view reports on current and historical peformance data in the Greenplum Performance Monitor graphical interface.

The database consists of three sets of tables, one set for current data, a “tail” set for data in transition, and another set for historical data.

Table 1.1 Greenplum Performance Monitor Data Tables

Current Data Table (External)	Tail Data Table (External)	Historical Data Table
system_now	*system_tail*	*system_history*
database_now	*database_tail*	*database_history*
queries_now	*queries_tail*	*queries_history*
iterators_now	*iterators_tail*	*iterators_history*

Current Data Tables

The current data is stored in flat files that are treated as external tables by the database. These files are located in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`.

For metrics, a row of data persists in the `metrics_now` table for one quantum period, after which it is replaced by the next row for that period. Query and iterator data persists in the current data tables until the query is completed, refreshing every quantum period.

Tail Data Tables

After passing out of the current data tables, all data is stored in the tail tables for a fixed period of two minutes before being committed to the historical data tables. These files are located in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`.



Note: Inside `$MASTER_DATA_DIRECTORY/gpperfmon/data/` you may see temporary files prefixed with an undercore character, such as `_queries now tail.dat`. Do not query or manipulate these temporary files, which may be safely ignored.

Historical Data Tables

The historical data tables are distributed Greenplum Database tables in the database `gpperfmon`. These tables are partitioned by month, allowing you to drop partitions for data that you no longer need. The installation utility creates eighteen months of partitioned tables. For more information on these tables and the Greenplum Performance Monitor database schema, see Appendix C, "Greenplum Performance Monitor Catalog Reference".

Greenplum Performance Monitor Console

Greenplum Performance Monitor provides a graphical interface for viewing performance metrics. With this browser-based application, you can create customized graphical views for tracking the performance of active queries and for analyzing historical data for queries and system metrics.

<: screen shot of GUI with callouts>

For more information, see Chapter 4, "Using Greenplum Performance Monitor".

Greenplum Performance Monitor Web Service

The Greenplum Performance Monitor Console queries the monitor database through a web service framework composed of a lightweight lighttpd web server and Python-based middleware. The lighttpd server is an open-source web server with a low memory footprint and light CPU load. For more information, see http://www.lighttpd.net/.

The monitor installation utility sets up the lighttpd web server and web service, prompting you for basic configuration information on the desired port and SSL options. Under normal conditions, the web server and web service API require minimal maintenance and administration, as described in "Web Server Administration" on page 18.

2. Installing Greenplum Performance Monitor

Installing Greenplum Performance Monitor involves the following high-level steps:

- Preparing for Installation
- Installing Greenplum Performance Monitor on the Master
- Installing the Segment Monitor Agents

This chapter also provides information about uninstalling Greenplum Performance Monitor.

Preparing for Installation

Before installing Greenplum Performance Monitor, create a monitor superuser and change settings in `postgresql.conf`.

You must be the Greenplum superuser (`gpadmin`) to install Greenplum Performance Monitor.

System Requirements

Requirements for the Greenplum Database master and segment hosts amply accomodate the requirements of Greenplum Performance Monitor. The monitor's demands on system resources depend on system load. In typical environments, the impact of Greenplum Performance Monitor on system resources is minimal.

Creating the Monitor Superuser

The Greenplum Performance Monitor requires a dedicated superuser role with database connection privileges. The monitor superuser owns the database that stores performance statistics data, connects to segment hosts to control agents, and performs other monitor-related operations.

To create the monitor superuser and assign connection privileges:

1. Using `psql` or another PostgreSQL-compatible client program, create a new superuser role with the `CREATEDB` attribute. For example:

   ```
   $ psql template1 -c 'CREATE ROLE gpmon WITH SUPERUSER
   CREATEDB LOGIN;'
   ```

2. Edit `$MASTER_DATA_DIRECTORY/pg_hba.conf` to give the `gpmon` superuser privileges to connect to the database. For example, to allow trusted local connections only, add the following entry to the file:

   ```
   local     all     gpmon     trust
   ```

3. Use `gpstop` with the `-u` option to reload `pg_hba.conf`:

   ```
   $ gpstop -u
   ```

Editing postgresql.conf

To enable the monitor in your Greenplum environment, you must change and verify `postgresql.conf` settings on all hosts as described in this section. For information about changing settings in a large database array, see "Setting a Local Configuration Parameter" in Chapter 19 of the *Greenplum Database Administrator Guide*.

To configure Greenplum Performance Monitor:

1. For each segment in the database array, edit `$MASTER_DATA_DIRECTORY/postgresql.conf` and add the following parameter, set to "on" as shown:

   ```
   gp_enable_gpperfmon=on
   ```

2. Find the following parameter in the "EXTERNAL TABLES" section of `postgresql.conf` and verify that it is set to "on" as shown:

   ```
   gp_external_enable_exec=on
   ```

3. If you want to use a different port than the default port 8888, add and set the following parameter:

   ```
   gpperfmon_port=<port_number>
   ```

4. Use `gpstop` with the `-u` option to reload `postgresql.conf`:

   ```
   $ gpstop -u
   ```



Note: The parameter `gp_enable_gpperfmon` is required for the collection of database, query and iterator data. The monitor can collect only system metrics when this parameter is not enabled.

Installing Greenplum Performance Monitor on the Master

Installing the monitor on the master host involves the following high-level tasks:

- Installing Greenplum Performance Monitor on the Master creates the installation directory on the host machine.
- Setting up Greenplum Performance Monitor on the Master creates and configures the the agents, the database, and other monitor components on the host machine.

Installing Greenplum Performance Monitor on the Master

This section describes how to create the installation directory for Greenplum Performance Monitor on your master host.

To install Greenplum Performance Monitor

1. Download or copy the installer file to the Greenplum Database master host. Installer files are available from Greenplum for RedHat (32-bit and 64-bit) and Solaris (64-bit) platforms.

2. Unzip the installer file where *PLATFORM* is either `RHEL4-i386` (RedHat 32-bit), `RHEL4-x86_64` (RedHat 64-bit) or `SOL-x86_64` (Solaris 64-bit). For example:

   ```
   # unzip greenplum-perfmon-2.0.0.0-PLATFORM.zip
   ```

3. Launch the installer using `bash`. For example:

   ```
   # /bin/bash greenplum-perfmon-2.0.0.0-PLATFORM.bin
   ```

4. The installer will prompt you to accept the Greenplum Performance Monitor license agreement. Type `yes` to accept the license agreement.

5. The installer will prompt you to provide an installation path. Press `ENTER` to accept the default install path (`/usr/local/greenplum-perfmon-2.0.0.0`), or enter an absolute path to an install location. You must have write permissions to the location you specify.

6. The installation directory contains the file `gpperfmon_path.sh` with path and environment settings for the monitor. You must source this file in this directory, or the `gpadmin` user's startup shell profile (such as `.bashrc`).

   For example, you could add a line similar to the following to your chosen profile files:

   ```
   source /usr/local/greenplum-perfmon-2.0.0.0/gpperfmon_path.sh
   ```

   After editing the chosen profile file, source it as the correct user to make the changes active. For example:

   ```
   $ source ~/.bashrc
   ```

   After sourcing `gpperfmon_path.sh`, you can continue to Setting up Greenplum Performance Monitor on the Master.

Setting up Greenplum Performance Monitor on the Master

The `gpperfmon` utility sets up Greenplum Performance Monitor on the master. This utility creates the following components on the master host:

- Dedicated Greenplum database for performance data (`gpperfmon`)
- Master monitor agent (`gpmmon`)
- Web server and scripts

During the setup process, the utility prompts you for values to configure these components. For more information on choosing values for your environment, see detailed descriptions of the configuration parameters in Appendix B, "Configuration Reference: gpperfmon.conf"

To accept the displayed default values for any of these parameters at configuration time, hit the `ENTER` key.

To set up Greenplum performance monitor on the master

1. Launch `gpperfmon` with the `--setup` option. For example:

   ```
   $ gpperfmon --setup
   ```

2. The utility prompts you to create the Greenplum Performance Monitor database. Type `Y` to agree and then type `Y` again to verify. The installer runs SQL scripts to create the database and displays a message indicating that the database was created successfully.
3. Enter a value for Quantum. The default is 15 seconds.
4. Enter a value for Minimum query run time. The default is 60.
5. Enter a value for Minimum detailed query run time. The default value displayed is the value you specified for Minimum query run time.
6. Enter a location for the Greenplum Performance Monitor log files. The default is `$MASTER_DATA_DIRECTORY/gpperfmon/logs`.
7. Enter the web server port for the monitor console. The default 28080.
8. (Optional) Enter `Y` or `N` to set up SSL connections for the monitor console. If you enter `Y`, you are prompted for the following DN information:

```
Country Name (2 letter code) [GB]:
State or Province Name (full name) [Berkshire]:
Locality Name (eg, city) [Newbury]:, s
Organization Name (eg, company) [My Company Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (eg, your name or your server's hostname) []:
Email Address []:
```



Note: Because database login information is sent over the network, Greenplum recommends setting up SSL to encrypt these sensitive communications.

When the utility displays a message indicating that `gpperfmon` configuration is complete, you have successfully installed Greenplum Performance Monitor on your master host.

For more information on next steps, see Installing the Segment Monitor Agents and Starting and Stopping Greenplum Performance Monitor.

Installing the Segment Monitor Agents

All segment agents must be installed in the same directory location as the installation directory on the master. For instance, if you installed to the default directory on the master, you must install all segment monitor agents to `/usr/local/greenplum-perfmon-2.0.0.0`.

To install Monitor Agents on the Segment Hosts

1. On the master host, create a tar file of your Greenplum Performance Monitor installation. For example (running as `root`):

```
# su -
```

```
# cd /usr/local
# gtar -cvf /usr/local/perfmon.tar greenplum-perfmon-2.0.0.0
```

2. Copy the tar file to the segment hosts using `gpscp`. For example:

   ```
   # gpscp -f /home/gpadmin/single_seg_hosts_file
   /usr/local/perfmon.tar =:/usr/local
   ```

3. Start an interactive session in `gpssh`. For example:

   ```
   # gpssh -f /home/gpadmin/single_seg_hosts_file
   ```

4. At the `gpssh` command prompt, untar the tar file in the installation directory on the segment hosts. This must be the same location as `gpperfmon` on your master host. For example:

   ```
   => gtar --directory /usr/local -xvf /usr/local/perfmon.tar
   ```

5. Confirm that the Greenplum Performance Monitor directory was installed in the correct location (the same location as `gpperfmon` on your master host). For example:

   ```
   => ls /usr/local/greenplum-perfmon-2.0.0.0
   ```

6. Change the ownership of the Greenplum Database install directory to the `gpadmin` user or group. For example:

   ```
   => chown -R gpadmin /usr/local/greenplum-perfmon-2.0.0.0
   => chgrp -R gpadmin /usr/local/greenplum-perfmon-2.0.0.0
   ```

7. Remove the tar file. For example:

   ```
   => rm /usr/local/gp.tar
   ```

   Once you have installed the master agent and all segment agents, you can start the monitor as descrbed in "Starting and Stopping Greenplum Performance Monitor" on page 15.

Uninstalling Greenplum Performance Monitor

To uninstall, you must remove monitor directories, drop the monitor database, and disable the monitor in your Greenplum Database configuration.

To uninstall Greenplum Performance Monitor

1. On the Greenplum Database master, open `postgresql.conf` and change the value of `gp_enable_gpperfmon` to `off`.

2. Reload the configuration file using the `gpstop` utility. For example:

   ```
   $gpstop -u
   ```

3. Remove the monitor installation directory. For example:

   ```
   $ rm -rf gpperfmon-3.2
   ```

4. Remove the directory that contains monitor data and configuration information. In a default setup this is `$MASTER_DATA_DIRECTORY/gpperfmon`. For example:

   ```
   $ rm -rf gpperfmon
   ```

5. Drop the monitor database using the `dropdb` utility. For example:

```
$ dropdb --echo gpperfmon
DROP DATABASE gpperfmon;
DROP DATABASE
```

6. Remove segment agents from the segment hosts. For example:

```
# gpssh -f /home/gpadmin/single_seg_hosts_file
=> cd /usr/local
=> rm -rf greenplum-perfmon-2.0.0.0
```

3 Administering Greenplum Performance Monitor

Much of the routine administration of Greenplum Performance Monitor is performed using the `gpperfmon` utility. This utility enables basic tasks such as stopping and restarting the monitor.

Other tasks for administration of the web server or the monitor database are performed with native tools or Greenplum utilities, as described in this chapter.

Starting and Stopping Greenplum Performance Monitor

The `gpperfmon` utility starts, stops and restarts Greenplum Performance Monitor with the following option flags:

- `--start`
- `--stop`
- `--restart`

For example, to start Greenplum Performance Monitor, run the following command:

```
$gpperfmon --start
```

These command options control the master agent, the segment agent processes, and the UI web server. If you want to exclude the UI web server from these commands, add the flag `--noweb`. For example, to restart agents without restarting the web server:

```
$gpperfmon --restart --noweb
```

For more information about `gpperfmon`, see "Utility Reference: gpperfmon" on page 26, or view usage information with the command `gpperfmon -?`.

Reconfiguring Greenplum Performance Monitor

Configuration parameters for Greenplum Performance Monitor are stored in `$MASTER_DATA_DIRECTORY/gpperfmon/conf/gpperfmon.conf`. This configuration file contains all the configuration options described in Appendix A, "Utility Reference: gpperfmon".

If you want to change configuration values for a monitor instance, edit `gpperfmon.conf`, or run `gpperfmon --setup`.

Editing the Configuration File

You can directly edit `gpperfmon.conf` to adjust a single value such as `min_detailed_query_time`.

To edit the configuration file

1. In a text editor, open `$MASTER_DATA_DIRECTORY/gpperfmon/conf/gpperfmon.conf`.
2. Locate the desired parameter and type the new value. For example, to change `min_detailed_query_time`, modify this setting found in the `[GPMMON]` section of the file:
   ```
   min_detailed_query_time=360
   ```
3. When you are finished making changes, save `gpperfmon.conf`.
4. Restart the monitor. For example:
   ```
   $ gpperfmon --restart
   ```

Reconfiguring a Monitor Instance with gpperfmon

Running `gpperfmon --setup` provides a convenient way to change one or all of the configuration values for a monitor instance. For each parameter prompt, the utility displays the existing value as the default.

Also, with `gpperfmon --setup` you can replace the existing monitor database with a fresh instance, or change values that are not available in `gpperfmon.conf`, such as the web server port and the DN values used for SSL.

To reconfigure a monitor instance with gpperfmon

1. Run `gpperfmon --setup`.
2. The utility displays the message warning that a monitor configuration file already exists. Type `Y` to reconfigure the monitor instance.
3. The utility prompts you to create a new database, warning you that all existing performance data will be lost. Type `Y` or `N` as desired.
4. Enter a value for Quantum.
5. Enter a value for Minimum query run time.
6. Enter a value for Minimum detailed query run time.
7. Enter a location for the Greenplum Performance Monitor log files.
8. Enter the web server port for the monitor console.
9. (Optional) Enter `Y` or `N` to set up SSL connections for the monitor console. If you enter `Y`, you are prompted for the following DN information:
   ```
   Country Name (2 letter code) [GB]:
   State or Province Name (full name) [Berkshire]:
   Locality Name (eg, city) [Newbury]:, s
   Organization Name (eg, company) [My Company Ltd]:
   Organizational Unit Name (eg, section) []:
   Common Name (eg, your name or your server's hostname) []:
   ```

```
Email Address []:
```



Note: Because database login information is sent over the network, Greenplum recommends setting up SSL to encrypt these sensitive communications.

When the utility displays a message indicating that `gpperfmon` configuration is complete, you have successfully reconfigured Greenplum Performance Monitor.

Agent Administration

This section describes basic agent administration tasks, including setting up a cronograph job to restart the master agent in case of failure, adding monitor hosts, and viewing agent log files.

Restarting the Master Agent

In this release, the master agent is not automatically restarted if it fails. Greenplum recommends setting up a cronograph job on the master host to start the master agent every few minutes. If the agent is already running, the cron job fails. If the agent is not running, the cron job restarts it.

Adding and Removing Monitored Hosts

Segment agents on new hosts are detected automatically by the master agent. To add a new monitored host, follow the procedure described in "Installing the Segment Monitor Agents" on page 12 . The master agent automatically detects, starts, and begins harvesting data from the segment agent.

Viewing and Maintaining Master Agent Log Files

Log messages for the master agent are written to `$MASTER_DATA_DIRECTORY/gpperfmon/logs/gpmmon.log` by default. To change the log file location, run `gpperfmon -- setup`, or edit the `log_location` parameter in `gpperfmon.conf`. For more information, see Reconfiguring Greenplum Performance Monitor.

If the parameter `verbose` is set to the default value of `0`, only fatal errors are logged. To log information for troubleshooting or debugging, set this value to 1 or 2 for detailed messages.

At higher logging levels, the size of the log files may grow dramatically. If the log files grow to an unmanageable size, you can restart the monitor to clear out existing monitor log messages.

Greenplum Performance Monitor Database Administration

As a distributed Greenplum database, the monitor database interoperates with the set of administration tools provided with your Greenplum installation. This section describes how to use Greenplum utilities in the routine maintenance of the monitor database.

With `gpperfmon --setup` you can replace the existing monitor database with a fresh instance. However, this process results in the loss of all existing performance data in the `gpperfmon` database.

Backing Up and Restoring the Monitor Database

You can back up and restore using the `gpdump` and `gprestore` utilities as described in chapter 21 of the *Greenplum Database Adminstrator Guide*, "Backing Up and Restoring Databases." This document can be found in the `$MASTER_DATA_DIRECTORY/gpperfmon//docs` directory of your Greenplum Database installation, or on the Greenplum Network.

Because the monitor database has a low number of tables, you may prefer to devise a backup plan using the table-level backup features of `gpdump`. For example, you can create scripts to run `gpdump` to back up the monthly partitions of the historical data tables on a monthly schedule. Alternately, you can back up your monitor database at the database level.

Maintaining Historical Data

The historical data tables are partitioned by month, allowing you to drop partitions for data that you no longer need. The `gpperfmon` installer creates eighteen months of partitioned tables. When that period is over, you may want to create new monthly partitions. For more information on partitioning Greenplum Database tables, see chapter 14 of the *Greenplum Database Adminstrator Guide*, "Defining Database Objects."

Web Server Administration

The lighttpd web server and web service middleware are installed in the `www` directory of your Greenplum Performance Monitor installation. For detailed information on lighttpd administration, see http://www.lighttpd.net/.

Starting and Stopping the Web Service

The `gpperfmon` utility starts, stops and restarts the lighttpd web server together with the monitor agents, using the following option flags:

```
--start
--stop
--restart
```

For example, to start the monitor and the web service:

```
gpperfmon --start
```

If you want to stop the monitor and run only the web service, use the `--noweb` option. For example:

```
gpperfmon --stop --noweb
```

Configuring the Web Server

The lighttpd web server's configuration is stored in `$MASTER_DATA_DIRECTORY/gpperfmon/conf/lighttpd.conf`. This configuration file contains the parameters that are set by the installation utility, including the web server port and DN information for optional SSL connections between the browser-based client and the web server.

`Web server port`

Specifies the port for the web server that provides the monitor console and web API.

SSL Setup Options

These options for SSL connections to the monitor console are set by `gpperfmon --setup`.

`Country Name`

Country name in format of a two-letter code. See ISO for a list of valid country codes.

`State or Province Name`

State or province name.

`Locality Name`

Locality name, such as city or town name.

`Organization Name`

Company name.

`Organizational Unit Name`

Section or function. For example, "system administration."

`Common Name`

Domain name. This value requires a server name to be entered for verification purposes. Entering anything other than a domain name yields an invalid certificate.

`Email Address`

A valid email address.

Viewing and Maintaining Web Server Log Files

Logs of web server access and server errors are written to `$MASTER_DATA_DIRECTORY/gpperfmon/logs/`. These two logs are:

- lighttpd-access.log
- lighttpd-error.log

<TBD: web.py log>

4 Using Greenplum Performance Monitor

Greenplum Performance Monitor provides immediate visibility into key areas affecting Greenplum Database performance. You can view system metrics such as CPU usage and network utilization. You can check the status of currently running queries, drilling down to iterator-level detail as desired. You can analyze historical data to identify trends or anomalies in the performance over time of the Greenplum Database system.

The browser-based console for Greenplum Performance Monitor provides an easy way to access performance data. This chapter gives information on getting started with the monitor console, as well as some guidance in the use of direct SQL queries and views on performance data.

About the Greenplum Performance Monitor Console

Greenplum Performance Monitor Console is supported for the following browsers with Macromedia Flash:

- Internet Explorer for Windows XP and Vista
- Mozilla Firefox for Windows and Linux
- Apple Safari for Macintosh

The monitor console is supported by a lighttpd web server installed on the master host. For more information about the web server, see "Web Server Administration" on page 18. The web service API for the monitor console is installed in `/www` under your installation directory.

To access the monitor console, open the following link a supported browser:

<TBD: example link to home/logon page, and/or procedure to logon with DB credentials, along with any details about the intial boostrap, if that is exposed to the user.>

Monitoring the Current System Workload

Greenplum Performance Monitor collects current system data at every `quantum` interval. These metrics include CPU load averages, memory and swap space usage, and network and disk usage. For a full listing of the metrics collected, see Appendix C, "Greenplum Performance Monitor Catalog Reference".

The monitor console provides quick access to the current system metrics through the <TBD: describe the right overview pane, with some details on drill-downs>.

Monitoring Active Queries

Greenplum Performance Monitor collects data on currently running queries at every `quantum` interval, updating the metrics at each interval until the query is completed. Current query metrics include the user running the query, the skew of CPU usage across segments, and detailed information for individual iterators executing the query plan. For a full listing of the metrics collected, see Appendix C, "Greenplum Performance Monitor Catalog Reference".

The monitor console provides quick access to the current system metrics through the <TBD: describe the left pane, again with drill downs>

<TBD: give a simple view of the database/queries/iterators now joined for key fields?>

Setting Thresholds for Notification

For selected system and query metrics, you can set thresholds for notification. When the specified thresholds are reached in the current database activity, the monitor console displays <TBD: or sends mail, or makes loud noises? Perhaps show a screen shot of visual cue. Confirm in UI testing.>

System metric thresholds are set by `gpperfmon --setup`, or by editing values in `gpperfmon.conf`. For more information on setting values for the available thresholds, see these parameters in the Greenplum Performance Monitor Console Options section of the configuration file reference:

- `load2_threshold`
- `cpu_threshold`
- `memory_threshold`
- `cpu_skew_threshold`

Viewing Historical System Metrics

Historical tables in the monitor database store all system data collected by the monitor. You can analyze this data to identify trends in the performance of the Greenplum Database system over time. For example, <TBD: break out the by-time period UI views in this section>

<TBD: example views, something to track trends over time periods.>

Viewing Aggregated System Metrics

With a default `quantum` setting of 15 seconds, the monitor collects very large amounts of historical system data. To view meaningful groupings of system data records, you can aggregate the data by selected time intervals with the monitor console, or with a SQL view like the one described in this section.

Viewing Aggregated System Metrics in the Monitor Console

<TBD: describe the GUI pages and steps to do this>

To view aggregated system metrics in the monitor console

1.

Viewing Aggregated System Metrics using SQL

This section provides an example SQL view for viewing aggregated system metrics. In this example view named *system_metrics_aggregate_1min*, rows of system metrics from *system_history* are averaged and aggregated by one minute intervals:

```
drop view if exists system_metrics_aggregate_1min;

create view system_metrics_aggregate_1min as (
    select date_trunc('minute', ctime) as sample_time,
    hostname,
    avg(mem_total) as mem_total, avg(mem_used) as mem_used,
    avg(mem_actual_used) as mem_actual_used,
    avg(mem_actual_free) as mem_actual_free,
    avg(swap_total) as swap_total, avg(swap_used) as swap_used,
    avg(swap_page_in) as swap_page_in,
    avg(swap_page_out) as swap_page_out,
    avg(cpu_user) as cpu_user, avg(cpu_sys) as cpu_sys,
    avg(cpu_idle) as cpu_idle,
    avg(load0) as load0, avg(load1) as load1,
    avg(load2) as load2,
    avg(disk_ro_rate) as disk_ro_rate,
    avg(disk_wo_rate) as disk_wo_rate,
    avg(disk_rb_rate) as disk_rb_rate,
    avg(disk_wb_rate) as disk_wb_rate,
    avg(net_rp_rate) as net_rp_rate,
    avg(net_wp_rate) as net_wp_rate,
    avg(net_rb_rate) as net_rb_rate,
    avg(net_wb_rate) as net_wb_rate

    from system_history

    group by hostname, sample_time

);
```

The aggregation interval is defined by creating a column *sample_time* from *system_history.ctime* and the Postgresql date function, `date_trunc`. Grouping the view by this column allows you to view data averaged within the minute-long value specified for `date_trunc`. Valid values are `microseconds`, `milliseconds`, `second`, `minute`, `hour`, `day`, `week`, `month`, `quarter`, `year`, `decade`, `century` and `millennium`.

Viewing Historical Query Metrics

Historical tables in the monitor database store all query and iterator data collected by the monitor. You can analyze this data to identify trends in the performance of Greenplum Database over time. <TBD: break out the by-time period views and the serious drilldowns into iterator information in this section>

<TBD: example views, something to track trends over time periods. Again, I think this would involve joining all three relevant tables, database, query and iterator>

Viewing Historical Database Metrics

Historical tables in the monitor database store statistics for the number of queries running and queued on the Greenplum Database array. You can analyze this data to identify trends in the performance of Greenplum Database over time.

<TBD: example views, something to track trends over time periods.>

Viewing Aggregated Database Statistics

The *database_history* table stores statistics for the number of queries running and queued on the Greenplum Database array. Calculating the average, minimum and maximum values for these statistics aggregated over time can yield a useful view of database utilization. This view is available in the monitor console <TBD:true?> or with a SQL view like the example shown in this section.

Viewing Aggregated Database Statistics in the Monitor Console

<TBD: describe the GUI pages and steps to do this>

To view aggregated database statistics in the monitor console

1.

Viewing Aggregated Database Statistics using SQL

This section provides an example SQL view for viewing aggregated database statistics for the system. In this example view named *database_metrics_1hour*, rows of statistics from the table *database_history* are aggregated by hour-long intervals:

```
create view database_metrics_1hour as (

    select date_trunc('hour', ctime) as sample_time,
    avg(queries_total) as queries_total,
    min(queries_total) as queries_total_min,
    max(queries_total) as queries_total_max,
    avg(queries_running) as queries_running,
    min(queries_running) as queries_running_min,
```

```
    max(queries_running) as queries_running_max,
    avg(queries_queued) as queries_queued,
    min(queries_queued) as queries_queued_min,
    max(queries_queued) as queries_queued_max

    from database_history

    group by sample_time

);
```

The aggregation interval is defined by creating a column *sample_time* from *database_history.ctime* and the Postgresql date function, `date_trunc`. For running, queued and total query values within the specified hour-long interval, Postgresql aggregate functions are applied to calculate average, minimum and maximum values.

A. Utility Reference: gpperfmon

This utility controls various aspects of the Greenplum Performance Monitor, including configuration and starting and stopping monitor agents.

Synopsis

```
gpperfmon [--setup] [--start] [--stop] [--restart] [--status]
[--version] [--noweb]
```

Description

The `gpperfmon` utility controls various aspects of Greenplum Performance Monitor. This utility configures the monitor components at installation time, starts and stops the monitor, and provides status and version information.

For starting and stopping the monitor, `gpperfmon` controls the master monitor agent, which passes commands to all segment agents. Commands for starting and stopping also control the web service, unless the --noweb options is specified.

Options

`--setup`

Configures monitor components on the master host. With this option, `gpperfmon` prompts for values to configure monitor components and writes the values to `gpperfmon.conf` and `lighttpd.conf`. For more information on these configuration parameters, see Appendix B, "Configuration Reference: gpperfmon.conf".

Running `gpperfmon --setup` after a successful installation effective resets all configuration values to the defaults. Use this option at installation time, or any time when you want to reconfigure all or most monitor parameters.

`--start`

Starts the web service and master agent, which in turn starts all segment monitor agents in the array. Can be used with the flag `--noweb` to start only the monitor agents without starting the web service.

`--stop`

Stops the web service and master agent, which in turn stops all segment monitor agents in the array. Can be used with the flag `--noweb` to stop only the monitor agents without stopping the web service.

`--restart`

Restarts the web service and master agent, which in turn restarts all segment monitor agents in the array. Can be used with the flag `--noweb` to restart only the monitor agents without restarting the web service.

This flag is used to reload configuration files after editing `gpperfmon.conf` or `lighttpd.conf`.

`--status`

Displays the status, either `Running` or `Stopped`, of the monitor and web service.

`--version`

Displays the version of the web service, master agent, and segment monitor agents.

`--noweb`

Used with `--start`, `--stop` and `--restart` to perform the command for the monitor agents only, excluding the web service.

Examples

Start the Greenplum Monitor master agent but not the web service:

```
gppperfmon --start --noweb
```

Start the utility in order to install or to change configuration values:

```
gpperfmon --setup
```

Check the status of the monitor and web service:

```
gpperfmon --status
```

See Also

Greenplum Performance Monitor Catalog Reference

B. Configuration Reference: gpperfmon.conf

Settings that determine the behavior of Greenplum Performance Monitor are stored in `$MASTER_DATA_DIRECTORY/gpperfmon/conf/gpperfmon.conf`. Any Greenplum Database user with write permissions to this directory can edit `gpperfmon.conf`. For configuration changes to take effect, you must save `gpperfmon.conf` and then restart the monitor with `gpperfmon --restart`.

Master Agent Options

These options for the master agent process `gpmmon` are set by `gpperfmon --setup`.

quantum

Specifies the time in seconds between updates from performance monitor agents on all segments. Defaults to 15 seconds, with valid values limited to the range between 10 and 60 seconds.

If you prefer a less granular view of performance, or want to collect and analyze minimal amounts of data for system metrics, choose a higher quantum. To collect data more frequently, choose a lower value.

min_query_time

Specifies the minimum query run time in seconds for statistics collection. The monitor logs all queries that run longer than this value in the `queries_history` table. For queries with shorter run times, no historical data is collected. Defaults to 60 seconds.

If you know that you want to collect data for all queries, you can set this parameter to a low value. Setting the minimum query run time to zero, however, collects data even for the numerous queries run by the monitor itself, creating a large amount of data that may not be useful.

min_detailed_query_time

Specifies the minimum iterator run time in seconds for statistics collection. The monitor logs all iterators that run longer than this value in the `iterators_history` table. For iterators with shorter run times, no data is collected. Minimum value is 10 seconds.

Defaults to the value of `min_query_time`, and must always be equal to, or greater than, the value of **`-m`**. Setting **`-d`** higher than **`-m`** allows you to log detailed iterator data only for especially complex, long-running queries, while still logging basic query data for shorter queries.

Given the complexity and size of iterator data, you may want to adjust this parameter according to the size of data collected. If the *iterators_history* tables are growing to excessive size without providing useful information, you can raise the value of this parameter to log iterator detail for fewer queries.

log_location

Specifies a directory location for monitor log files. Default is `$MASTER_DATA_DIRECTORY/gpperfmon/logs`.

verbose

Verbose mode shows progress and status messages of the performance tests as they are run. The default is `0`, which displays and logs no progress or status messages. Valid values of `1` and `2` display increasingly detailed messages, and are recommended only for troubleshooting purposes. These higher settings cause dramatic growth in log file size.

Greenplum Performance Monitor Console Options

These options for the monitor console and web server are stored in `gpperfmon.conf`. The utility command `gpperfmon --setup` sets most, but not all, of these options. For more information, see "Setting up Greenplum Performance Monitor on the Master" on page 11.

The related parameter `web_server_port` is stored in `httpd.conf`. For more information about this parameter, see "Web Server Administration" on page 18.

max_data_points

Specifies the maximum number of data points to display in the monitor console in order to prevent returning data sets of unmanageable size. Default is 600. When data sets larger than this value are requested, the data is automatically aggregated and reduced to more usable units for display in the monitor console.

load2_threshold

Sets the threshold to highlight *load2* metrics in the monitor console. The *load2* column reflects the CPU load average for the fifteen-minute period prior to metric collection. Default is `2`.

cpu_threshold

Sets the threshold to highlight CPU usage metrics in the monitor console. Default is `80`.

memory_threshold

Sets the threshold to highlight memory usage metrics in the monitor console. Default is `80`.

cpu_skew_threshold

Sets the threshold to highlight skew metrics in the monitor console.

remote

Determines whether the monitor console accesses current data files (such as *system_now* or *queries_now*) directly on the file system, or through the Greenplum Database. Direct access to the file system has a significant performance advantage, but some firewall and security configurations may require database connections.

The default value of `no | False` allows direct access to the file system. Changing this parameter to `yes | True` forces the monitor console to connect to the database to query current data tables.

C. Greenplum Performance Monitor Catalog Reference

This is a reference of the system catalog tables for Greenplum Performance Monitor.

The monitor database stores each performance statistic type (system metrics, database information, queries, and iterators) in three separate sets of tables: one for current statistics, one for "tail" data in transition, and one for historical data.

- system_now
- system_tail
- system_history
- database_now
- database_tail
- database_history
- queries_now
- queries_tail
- queries_history
- iterators_now
- iterators_tail
- iterators_history

system_now

The table *system_now* contains current system utilization metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current system utilization data is stored in *system_now* during the period between data collection from the monitor agents and automatic commitment to the *system_history* table.

Table C.1 gpperfmon.system_now

column	type	description
ctime	timestamp	Time this row was created.
hostname	varchar(64)	Hostname of the system to which the metric belongs.
mem_total	bigint	Total system memory for the system to which the metric belongs, including actual memory and swap space.
mem_used	bigint	Used system memory for the system to which the metric belongs.
mem_actual_used	bigint	Used actual memory (memory not including portion reserved for cache and buffers).
mem_actual_free	bigint	Free actual memory (memory not including portion reserved for cache and buffers).
swap_total	bigint	Total swap space for the system to which the metric belongs.
swap_used	bigint	Used swap space for the system to which the metric belongs.
swap_page_in	bigint	Number of swap pages in.
swap_page_out	bigint	Number of swap pages out.
cpu_user	float	CPU usage by the user.
cpu_sys	float	CPU usage for the system.
cpu_idle	float	Idle CPU capacity at metric collection time.
load0	float	CPU load average for the prior one-minute period.
load1		CPU load average for the prior five-minute period.
load2		CPU load average for the prior fifteen-minute period.
quantum	int	Interval between metric collection for this metric entry.
disk_ro_rate	bigint	Disk read operations per second.
disk_wo_rate	bigint	Disk write operations per second.
disk_rb_rate	bigint	Bytes per second for disk read operations.
disk_wb_rate	bigint	Bytes per second for disk write operations.
net_rp_rate	bigint	Packets per second on the system network for read operations.
net_wp_rate	bigint	Packets per second on the system network for write operations.

Table C.1 gpperfmon.system_now

column	type	description
net_rb_rate	bigint	Bytes per second on the system network for read operations.
net_wb_rate	bigint	Bytes per second on the system network for write operations.

system_tail

The table *system_tail* contains system utilization metrics queued for commitment to *system_history.* This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. System utilization data is stored in *system_tail* between flushing from *system_now* and automatic commitment to *system_history*.

Table C.1 gpperfmon.system_now

column	type	description
ctime	timestamp	Time this row was created.
hostname	varchar(64)	Hostname of the system to which the metric belongs.
mem_total	bigint	Total system memory for the system to which the metric belongs, including actual memory and swap space.
mem_used	bigint	Used system memory for the system to which the metric belongs.
mem_actual_used	bigint	Used actual memory (memory not including portion reserved for cache and buffers).
mem_actual_free	bigint	Free actual memory (memory not including portion reserved for cache and buffers).
swap_total	bigint	Total swap space for the system to which the metric belongs.
swap_used	bigint	Used swap space for the system to which the metric belongs.
swap_page_in	bigint	Number of swap pages in.
swap_page_out	bigint	Number of swap pages out.
cpu_user	float	CPU usage by the user.
cpu_sys	float	CPU usage for the system.
cpu_idle	float	Idle CPU capacity at metric collection time.
load0	float	CPU load average for the prior one-minute period.
load1		CPU load average for the prior five-minute period.
load2		CPU load average for the prior fifteen-minute period.
quantum	int	Interval between metric collection for this metric entry.
disk_ro_rate	bigint	Disk read operations per second.
disk_wo_rate	bigint	Disk write operations per second.
disk_rb_rate	bigint	Bytes per second for disk read operations.
disk_wb_rate	bigint	Bytes per second for disk write operations.
net_rp_rate	bigint	Packets per second on the system network for read operations.
net_wp_rate	bigint	Packets per second on the system network for write operations.

Table C.1 gpperfmon.system_now

column	type	description
net_rb_rate	bigint	Bytes per second on the system network for read operations.
net_wb_rate	bigint	Bytes per second on the system network for write operations.

system_history

The table *system_history* stores historical system utilization metrics. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

Table C.1 gpperfmon.system_history

column	type	description
ctime	timestamp	Time this row was created.
hostname	varchar (64)	Hostname of the system to which the metric belongs.
mem_total	bigint	Total system memory for the system to which the metric belongs.
mem_used	bigint	Used system memory for the system to which the metric belongs.
mem_actual_used	bigint	Used actual memory (memory not including portion reserved for cache and buffers).
mem_actual_free	bigint	Free actual memory (memory not including portion reserved for cache and buffers).
swap_total	bigint	Total swap space for the system to which the metric belongs.
swap_used	bigint	Used swap space for the system to which the metric belongs.
swap_page_in	bigint	Number of swap pages in.
swap_page_out	bigint	Number of swap pages out.
cpu_user	float	CPU usage by the user.
cpu_sys	float	CPU usage for the system.
cpu_idle	float	Idle CPU capacity at metric collection time.
load0	float	CPU load average for the prior one-minute period.
load1	float	CPU load average for the prior five-minute period.
load2	float	CPU load average for the prior fifteen-minute period.
quantum	int	Interval between metric collection for this metric entry.
disk_ro_rate	bigint	Disk read operations per second.
disk_wo_rate	bigint	Disk write operations per second.
disk_rb_rate	bigint	Bytes per second for disk read operations.
disk_wb_rate	bigint	Bytes per second for disk write operations.
net_rp_rate	bigint	Packets per second on the system network for read operations.
net_wp_rate	bigint	Packets per second on the system network for write operations.

Table C.1 gpperfmon.system_history

column	type	description
net_rb_rate	bigint	Bytes per second on the system network for read operations.
net_wb_rate	bigint	Bytes per second on the system network for write operations.

database_now

The table *database_now* contains database-wide query information for current queries. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current database-wide query data is stored in *database_now* for each quantum period, after which is passes to *database_tail*..

Table C.1 gpperfmon.iterators_history

column	type	description
ctime	timestamp	Time this row was created.
queries_total	int	The total number of queries.
queries_running	int	The number of queries currently running.
queries_queued	int	The number of queries queued.

database_tail

The table *database_tail* contains database-wide query information for current queries. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current database-wide query data is stored in *database_tail* between flushing from *database_now* and automatic commitment to the *database_history* table..

Table C.1 gpperfmon.iterators_history

column	type	description
ctime	timestamp	Time this row was created.
queries_total	int	The total number of queries.
queries_running	int	The number of queries currently running.
queries_queued	int	The number of queries queued.

database_history

The table *database_history* contains historical database-wide query information. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need..

Table C.1 gpperfmon.iterators_history

column	type	description
ctime	timestamp	Time this row was created.
queries_total	int	The total number of queries.
queries_running	int	The number of queries running at data collection time for this row.
queries_queued	int	The number of queries queued at data collection time for this row.

queries_now

The table *queries_now* contains currently running query status information. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current query status data is stored in *quieries_now* until the query is completed, refreshing every quantum period. When the query is completed, the query data is passed into *queries_tail*.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

Table C.1 gpperfmon.queries_now

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`.
uname	varchar(64)	Username that issued this query.
db	varchar(64)	Name of the database queried.
tsubmit	timestamp	Time the query was submitted.
tstart	timestamp	Time the query was started.
tfin	timestamp	Time the query finished. For running queries, this value is always zero.
status	varchar(64)	Status of the query -- `start`, `done`, or `abort`.
rows_out	bigin	Rows out for the query.
cpu_elapsed	bigin	CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array.
cpu_currpct	float	Current CPU percent average for all processes executing this query. The percentages for all processes running on each segment are averaged, and then the average of all those values is calculated to render this metric.
skew_cpu	float	Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
skew_rows	float	Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
query_hash	bigint	This column is not implemented at time of writing.
query_text	text	Query text for this query.
query_plan	text	This column is not implemented at time of writing.

queries_tail

The table *queries_tail* contains currently running query status information. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current query status data is stored in *queries_tail* between flushing from *queries_now* and automatic commitment to the *queries_history* table.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

Table C.1 gpmon_2.queries_now

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`.
uname	varchar(64)	Username that issued this query.
db	varchar(64)	Name of the database queried.
tsubmit	timestamp	Time the query was submitted.
tstart	timestamp	Time the query was started.
tfin	timestamp	Time the query finished. For running queries, this value is always zero.
status	varchar(64)	Status of the query -- `start`, `done`, or `abort`.
rows_out	bigin	Rows out for the query.
cpu_elapsed	bigin	CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array.
cpu_currpct	float	Current CPU percent average is always zero in queries_tail.
skew_cpu	float	Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
skew_rows	float	Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
query_hash	bigint	This column is not implemented at time of writing.
query_text	text	Query text for this query.
query_plan	text	This column is not implemented at time of writing.

queries_history

The table *queries_history* stores historical query list status. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

Table C.1 gpperfmon.queries_history

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`.
username	varchar(64)	Username that issued this query.
db	varchar(64)	Name of the database queried.
tsubmit	timestamp	Time the query was submitted.
tstart	timestamp	Time the query was started.
tfinish	timestamp	Time the query finished.
status	varchar(64)	Status of the query -- `start`, `done`, or `abort`.
rows_out	bigin	Rows out for the query.
cpu_elapsed	bigin	CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array.
cpu_currpct	float	Current CPU percent average is always zero in historical data.
skew_cpu	float	Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
skew_rows	float	Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
query_hash	bigint	This column is not implemented at time of writing.
query_text	text	Query text for this query.
query_plan	text	This column is not implemented at time of writing.

iterators_now

The table *iterators_now* contains current query iterators and their metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current iterator data is stored in *iterators_now* until the query is completed, refreshing every quantum period. When the query is completed, the detailed iterator data is passed into *iterators_tail*.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan. By looking at the metrics of the individual operations in a query plan, you can determine the status of a query's execution.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

Table C.1 gpperfmon.iterators_now

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`.
segid	int	The segment ID (`dbid` from `gp_configuration`).
pid	int	The process ID of the `postgres` process executing this iterator.
nid	int	The ID of this query plan node from the slice plan.
pnid	int	The ID of the parent query plan node from the slice plan.
hostname	varchar(64)	Hostname for the system where the iterator is executed.
ntype	varchar(64)	The query plan node operation type. Possible values are: `AdaptiveNestLoopJoin, HashAgg, Append, AppendOnlyScan, BitmapAnd, BitmapHeapScan, BitmapIndexScan, BitmapOr, ExternalScan, FunctionScan, Group, Hash, HashJoin, IndexScan, Limit, Materialize, MergeJoin, Motion, NestLoopJoin, ResultNode, SequentialScan, SetOpNode, ShareInputScan, Sort, SubqueryScan, TidScan, Unique, ValuesScan, Window.`
nstatus	varchar(64)	The status of this query plan node operation. Possible values are: `Initialize, Executing and Finished.`
tstart	timestamp	Start time for the iterator.
tduration	int	Duration of the execution.
pmemsize	bigint	Memory size from Greenplum Database memory allocation data.

Table C.1 gpperfmon.iterators_now

column	type	description
pmemmax	bigint	Maximum memory from Greenplum Database memory allocation data.
memsize	bigint	Memory size from OS memory allocation data.
memresid	bigint	Resident memory, as opposed to shared or virtual memory.
memshare	bigint	Shared memory figure from OS memory allocation data.
cpu_elapsed	bigint	Total CPU usage of the process executing the iterator.
cpu_currpct	float	CPU percentage for the process executing the iterator.
phase	varchar(64)	This column is not implemented at time of writing.
rowsout	bigint	Number of rows output by the iterator.
rowsout_est	bigint	The query planner's estimate of rows output by the iterator.
m0_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`.
m0_unit	varchar(64)	The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`.
m0_val	bigint	The value of this metric.
m0_est	bigint	The estimated value of this metric.

m1_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Tuples`
		`Append - Append Current Input Source`
		`AppendOnlyScan - AppendOnlyScan Rescan`
		`BitmapHeapScan - BitmapHeapScan Pages`
		`Hash - Hash Spill Batches`
		`HashJoin - HashJoin Spill Batches`
		`IndexScan - IndexScan Restore Pos`
		`MergeJoin - MergeJoin Inner Side Tuples`
		`NestLoopJoin - NestLoopJoin Inner Tuples`
		`SequentialScan - SeqScan Page Stats`
		`ShareInputScan - ShareInputScan Restore Pos`
		`Sort - Sort Memory Usage`
		`SubqueryScan - SubqueryScan Rescan`

Table C.1 gpperfmon.iterators_now

column	type	description

m1_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Append - Inputs`
		`AppendOnlyScan - Rescans`
		`BitmapHeapScan - Pages`
		`Hash - Batches`
		`HashJoin - Batches`
		`IndexScan - Restores`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Pages`
		`ShareInputScan - Restores`
		`Sort - Bytes`
		`SubqueryScan - Rescans`

m1_val	bigint	The value of this metric.
m1_est	bigint	The estimated value of this metric.

m2_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Bytes`
		`Hash - Hash Spill Tuples`
		`HashJoin - HashJoin Spill Tuples`
		`IndexScan - IndexScan Rescan`
		`MergeJoin - MergeJoin Outer Side Tuples`
		`NestLoopJoin - NestLoopJoin Outer Tuples`
		`SequentialScan - SeqScan Restore Pos`
		`ShareInputScan - ShareInputScan Rescan`
		`Sort - Sort Spill Tuples`

m2_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Hash - Tuples`
		`HashJoin - Tuples`
		`IndexScan - Rescans`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Restores`
		`ShareInputScan - Rescans`
		`Sort - Tuples`

m2_val	bigint	The value of this metric.
m2_est	bigint	The estimated value of this metric.

Table C.1 gpperfmon.iterators_now

column	type	description

m3_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Batches`
		`Hash - Hash Spill Bytes`
		`HashJoin - HashJoin Spill Bytes`
		`SequentialScan - SeqScan Rescan`
		`Sort - Sort Spill Bytes`

m3_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Batches`
		`Hash - Bytes`
		`HashJoin - Bytes`
		`SequentialScan - Rescans`
		`Sort - Bytes`

m3_val	bigint	The value of this metric.
m3_est	bigint	The estimated value of this metric.

m4_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Pass`
		`Sort - Sort Spill Pass`

m4_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Passes`
		`Sort - Passes`

m4_val	bigint	The value of this metric.
m4_est	bigint	The estimated value of this metric.

m5_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Tuples`
		`Sort - Sort Current Spill Pass Tuples`

m5_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Sort - Tuples`

m5_val	bigint	The value of this metric.
m5_est	bigint	The estimated value of this metric.

m6_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Bytes`
		`Sort - Sort Current Spill Pass Bytes`

Table C.1 gpperfmon.iterators_now

column	type	description

m6_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Sort - Bytes`

m6_val	bigint	The value of this metric.
m6_est	bigint	The estimated value of this metric.
m7_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Tuples`
m7_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Tuples`
m7_val	bigint	The value of this metric.
m7_est	bigint	The estimated value of this metric.
m8_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Bytes`
m8_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Bytes`
m8_val	bigint	The value of this metric.
m8_est	bigint	The estimated value of this metric.
m9_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Batch`
m9_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Batches`
m9_val	bigint	The value of this metric.
m9_est	bigint	The estimated value of this metric.

iterators_tail

The table *iterators_tail* contains current query iterators and their metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current iterator data is stored in *iterators_tail* between flushing from *iterators_now* and automatic commitment to *iterators_history*.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan. By looking at the metrics of the individual operations in a query plan, you can determine the status of a query's execution.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

Table C.1 gpmon_2.iterators_now

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`.
segid	int	The segment ID (`dbid` from `gp_configuration`).
pid	int	The process ID of the `postgres` process executing this iterator.
nid	int	The ID of this query plan node from the slice plan.
pnid	int	The ID of the parent query plan node from the slice plan.
hostname	varchar(64)	Hostname for the system where the iterator is executed.
ntype	varchar(64)	The query plan node operation type. Possible values are: `AdaptiveNestLoopJoin, HashAgg, Append, AppendOnlyScan, BitmapAnd, BitmapHeapScan, BitmapIndexScan, BitmapOr, ExternalScan, FunctionScan, Group, Hash, HashJoin, IndexScan, Limit, Materialize, MergeJoin, Motion, NestLoopJoin, ResultNode, SequentialScan, SetOpNode, ShareInputScan, Sort, SubqueryScan, TidScan, Unique, ValuesScan, Window.`
nstatus	varchar(64)	The status of this query plan node operation. Possible values are: `Initialize, Executing and Finished.`
tstart	timestamp	Start time for the iterator.
tduration	int	Duration of the execution.
pmemsize	bigint	Memory size from Greenplum Database memory allocation data.
pmemmax	bigint	Maximum memory from Greenplum Database memory allocation data.

Table C.1 gpmon_2.iterators_now

column	type	description
memsize	bigint	Memory size from OS memory allocation data.
memresid	bigint	Resident memory, as opposed to shared or virtual memory.
memshare	bigint	Shared memory figure from OS memory allocation data.
cpu_elapsed	bigint	Total CPU usage of the process executing the iterator.
cpu_currpct	float	CPU percentage for the process executing the iterator.
phase	varchar(64)	This column is not implemented at time of writing.
rowsout	bigint	Number of rows output by the iterator.
rowsout_est	bigint	The query planner's estimate of rows output by the iterator.
m0_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`.
m0_unit	varchar(64)	The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`.
m0_val	bigint	The value of this metric.
m0_est	bigint	The estimated value of this metric.

m1_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Tuples`
		`Append - Append Current Input Source`
		`AppendOnlyScan - AppendOnlyScan Rescan`
		`BitmapHeapScan - BitmapHeapScan Pages`
		`Hash - Hash Spill Batches`
		`HashJoin - HashJoin Spill Batches`
		`IndexScan - IndexScan Restore Pos`
		`MergeJoin - MergeJoin Inner Side Tuples`
		`NestLoopJoin - NestLoopJoin Inner Tuples`
		`SequentialScan - SeqScan Page Stats`
		`ShareInputScan - ShareInputScan Restore Pos`
		`Sort - Sort Memory Usage`
		`SubqueryScan - SubqueryScan Rescan`

Table C.1 gpmon_2.iterators_now

column	type	description

m1_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Append - Inputs`
		`AppendOnlyScan - Rescans`
		`BitmapHeapScan - Pages`
		`Hash - Batches`
		`HashJoin - Batches`
		`IndexScan - Restores`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Pages`
		`ShareInputScan - Restores`
		`Sort - Bytes`
		`SubqueryScan - Rescans`

m1_val	bigint	The value of this metric.
m1_est	bigint	The estimated value of this metric.

m2_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Bytes`
		`Hash - Hash Spill Tuples`
		`HashJoin - HashJoin Spill Tuples`
		`IndexScan - IndexScan Rescan`
		`MergeJoin - MergeJoin Outer Side Tuples`
		`NestLoopJoin - NestLoopJoin Outer Tuples`
		`SequentialScan - SeqScan Restore Pos`
		`ShareInputScan - ShareInputScan Rescan`
		`Sort - Sort Spill Tuples`

m2_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Hash - Tuples`
		`HashJoin - Tuples`
		`IndexScan - Rescans`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Restores`
		`ShareInputScan - Rescans`
		`Sort - Tuples`

m2_val	bigint	The value of this metric.
m2_est	bigint	The estimated value of this metric.

Table C.1 gpmon_2.iterators_now

column	type	description

m3_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Batches`
		`Hash - Hash Spill Bytes`
		`HashJoin - HashJoin Spill Bytes`
		`SequentialScan - SeqScan Rescan`
		`Sort - Sort Spill Bytes`

m3_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Batches`
		`Hash - Bytes`
		`HashJoin - Bytes`
		`SequentialScan - Rescans`
		`Sort - Bytes`

m3_val	bigint	The value of this metric.
m3_est	bigint	The estimated value of this metric.

m4_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Pass`
		`Sort - Sort Spill Pass`

m4_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Passes`
		`Sort - Passes`

m4_val	bigint	The value of this metric.
m4_est	bigint	The estimated value of this metric.

m5_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Tuples`
		`Sort - Sort Current Spill Pass Tuples`

m5_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Sort - Tuples`

m5_val	bigint	The value of this metric.
m5_est	bigint	The estimated value of this metric.

m6_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Bytes`
		`Sort - Sort Current Spill Pass Bytes`

Table C.1 gpmon_2.iterators_now

column	type	description

m6_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Sort - Bytes`

m6_val	bigint	The value of this metric.
m6_est	bigint	The estimated value of this metric.
m7_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Tuples`
m7_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Tuples`
m7_val	bigint	The value of this metric.
m7_est	bigint	The estimated value of this metric.
m8_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Bytes`
m8_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Bytes`
m8_val	bigint	The value of this metric.
m8_est	bigint	The estimated value of this metric.
m9_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Batch`
m9_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Batches`
m9_val	bigint	The value of this metric.
m9_est	bigint	The estimated value of this metric.

iterators_history

The table *iterators_history* contains historical query iterators and their metrics. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

Table C.1 gpperfmon.iterators_history

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`.
segid	int	The segment ID (`dbid` from `gp_configuration`).
pid	int	The process ID of the `postgres` process executing this iterator.
nid	int	The ID of this query plan node from the slice plan.
pnid	int	The ID of the parent query plan node from the slice plan.
hostname	varchar(64)	Hostname for the system where the iterator is executed.
ntype	varchar(64)	The query plan node operation type. Possible values are: `AdaptiveNestLoopJoin, HashAgg, Append, AppendOnlyScan, BitmapAnd, BitmapHeapScan, BitmapIndexScan, BitmapOr, ExternalScan, FunctionScan, Group, Hash, HashJoin, IndexScan, Limit, Materialize, MergeJoin, Motion, NestLoopJoin, ResultNode, SequentialScan, SetOpNode, ShareInputScan, Sort, SubqueryScan, TidScan, Unique, ValuesScan, Window.`
nstatus	varchar(64)	The status of this query plan node operation. Possible values are: `Initialize, Executing and Finished.`
tstart	timestamp	Start time for the iterator.
tduration	int	Duration of the execution.
pmemsize	bigint	Memory size from Greenplum Database memory allocation data.
pmemmax	bigint	Maximum memory from Greenplum Database memory allocation data.

Table C.1 gpperfmon.iterators_history

column	type	description
memsize	bigint	Memory size from OS memory allocation data.
memresid	bigint	Resident memory, as opposed to shared or virtual memory.
memshare	bigint	Shared memory figure from OS memory allocation data.
cpu_elapsed	bigint	Total CPU usage of the process executing the iterator.
cpu_currpct	float	This value is always zero for historical iterators.
phase	varchar(64)	This column is not implemented at time of writing.
rowsout	bigint	Number of rows output by the iterator.
rowsout_est	bigint	The query planner's estimate of rows output by the iterator.
m0_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`.
m0_unit	varchar(64)	The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`.
m0_val	bigint	The value of this metric.
m0_est	bigint	The estimated value of this metric.

m1_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Tuples`
		`Append - Append Current Input Source`
		`AppendOnlyScan - AppendOnlyScan Rescan`
		`BitmapHeapScan - BitmapHeapScan Pages`
		`Hash - Hash Spill Batches`
		`HashJoin - HashJoin Spill Batches`
		`IndexScan - IndexScan Restore Pos`
		`MergeJoin - MergeJoin Inner Side Tuples`
		`NestLoopJoin - NestLoopJoin Inner Tuples`
		`SequentialScan - SeqScan Page Stats`
		`ShareInputScan - ShareInputScan Restore Pos`
		`Sort - Sort Memory Usage`
		`SubqueryScan - SubqueryScan Rescan`

Table C.1 gpperfmon.iterators_history

column	type	description

m1_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Append - Inputs`
		`AppendOnlyScan - Rescans`
		`BitmapHeapScan - Pages`
		`Hash - Batches`
		`HashJoin - Batches`
		`IndexScan - Restores`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Pages`
		`ShareInputScan - Restores`
		`Sort - Bytes`
		`SubqueryScan - Rescans`

m1_val	bigint	The value of this metric.
m1_est	bigint	The estimated value of this metric.

m2_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Bytes`
		`Hash - Hash Spill Tuples`
		`HashJoin - HashJoin Spill Tuples`
		`IndexScan - IndexScan Rescan`
		`MergeJoin - MergeJoin Outer Side Tuples`
		`NestLoopJoin - NestLoopJoin Outer Tuples`
		`SequentialScan - SeqScan Restore Pos`
		`ShareInputScan - ShareInputScan Rescan`
		`Sort - Sort Spill Tuples`

m2_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Hash - Tuples`
		`HashJoin - Tuples`
		`IndexScan - Rescans`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Restores`
		`ShareInputScan - Rescans`
		`Sort - Tuples`

m2_val	bigint	The value of this metric.
m2_est	bigint	The estimated value of this metric.

Table C.1 gpperfmon.iterators_history

column	type	description

m3_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Batches`
		`Hash - Hash Spill Bytes`
		`HashJoin - HashJoin Spill Bytes`
		`SequentialScan - SeqScan Rescan`
		`Sort - Sort Spill Bytes`

m3_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Batches`
		`Hash - Bytes`
		`HashJoin - Bytes`
		`SequentialScan - Rescans`
		`Sort - Bytes`

m3_val	bigint	The value of this metric.
m3_est	bigint	The estimated value of this metric.

m4_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Pass`
		`Sort - Sort Spill Pass`

m4_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Passes`
		`Sort - Passes`

m4_val	bigint	The value of this metric.
m4_est	bigint	The estimated value of this metric.

m5_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Tuples`
		`Sort - Sort Current Spill Pass Tuples`

m5_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Sort - Tuples`

m5_val	bigint	The value of this metric.
m5_est	bigint	The estimated value of this metric.

m6_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Bytes`
		`Sort - Sort Current Spill Pass Bytes`

Table C.1 gpperfmon.iterators_history

column	type	description

m6_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Sort - Bytes`

m6_val	bigint	The value of this metric.
m6_est	bigint	The estimated value of this metric.
m7_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Tuples`
m7_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Tuples`
m7_val	bigint	The value of this metric.
m7_est	bigint	The estimated value of this metric.
m8_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Bytes`
m8_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Bytes`
m8_val	bigint	The value of this metric.
m8_est	bigint	The estimated value of this metric.
m9_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Batch`
m9_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Batches`
m9_val	bigint	The value of this metric.
m9_est	bigint	The estimated value of this metric.

Index

A

active queries: 22
agent
  gpmmon: 6
  gpsmon: 6
aggregating data
  query metrics: 24
  system metrics: 22, 23
alerts and notification: 22
architecture: 5
authentication: 12, 16

B

bashrc: 11
browser support: 21

C

components: 5, 11, 26
configuration: 10, 11, 26
  Greenplum Database: 10
connections
  Greenplum Performance Monitor console: 12, 29
console: 7, 12, 22, 29
Country Name: 19
cpu_threshold: 29
current data
  queries: 22
current data tables: 29
current query data: 22
current system data: 21

D

data analysis: 7
data tables
  current data: 6
  historical: 7
  tail: 7
database
  monitor database: 6
database utilization: 24
database_history: 24
date_trunc function: 23, 24
disable monitor: 13
DN: 12, 16

E

enable monitor: 10

F

files, installer: 10

G

gp_enable_gpperfmon: 10
gp_external_enable_exec: 10
gpmmon: 28
gpmon user: 9
gpperfmon --setup: 22
gpperfmon.conf: 11, 15, 22, 26
gpperfmon_path.sh: 11
gpperfmon_port: 10
graphical user interface: 7
Greenplum Database configuration: 10
Greenplum Performance Monitor Console: 7
  browser support: 21
GUI: 7, 21

H

historical data: 7
  query metrics: 24
  system metrics: 22, 23
historical query metrics: 24
historical system metrics: 22, 23

I

installation path: 10, 11, 12
installing
  on master host: 10
  segment host: 12
interconnect: 5
iterators: 22

L

lighttpd: 21
lighttpd.conf: 11, 26
load2_threshold: 29
log_location: 29

M

max_data_points: 29
memory_threshold: 29
min_detailed_query_time: 28
min_query_time: 28
monitor user: 9

N

notification: 22

P

partitioning: 7
path, installlation: 10
platform support: 10
port: 10

web server: 12, 16
Postgresql functions
avg: 24
date_func: 23, 24
max: 24
min: 24
postgresql.conf: 10
preinstallation: 9
process
gpmmon: 6
gpsmon: 6
psql: 6

Q

quantum: 6, 28
queries: 22
query
current data tables: 29
query plan: 22

R

reconfiguring: 15
RedHat: 10
remote: 29
removing the monitor: 13
role, gpmon: 9

S

secure communications: 12, 16
segment agent installation: 12
setup: 15
Solaris: 10
SQL view: 23, 24
SSL: 7, 12, 16
superuser: 9
supported browsers: 21
supported platforms: 10
system data: 21
system metrics: 10
system requirements: 9
system workload: 21
system_history: 23

T

tables
current data: 6, 29
database_history: 24
external tables: 29
historical data: 7
system_history: 23
tail: 7
tail tables: 7
thresholds: 22

U

uninstalling: 13
user interface: 7, 21, 29
user interface options: 22

V

verbose: 29
views: 23, 24

W

web server: 7
web server port: 12, 16, 19
web service API: 21
web sever: 21